SOFTWARE-DEFINED SYSTEMS FINAL PROJECT

SEP 29 — NOV 16 — NOV 17

2110415 SEMESTER 1/2026

KUNWADEE SRIPANIDKULCHAI, PH.D.

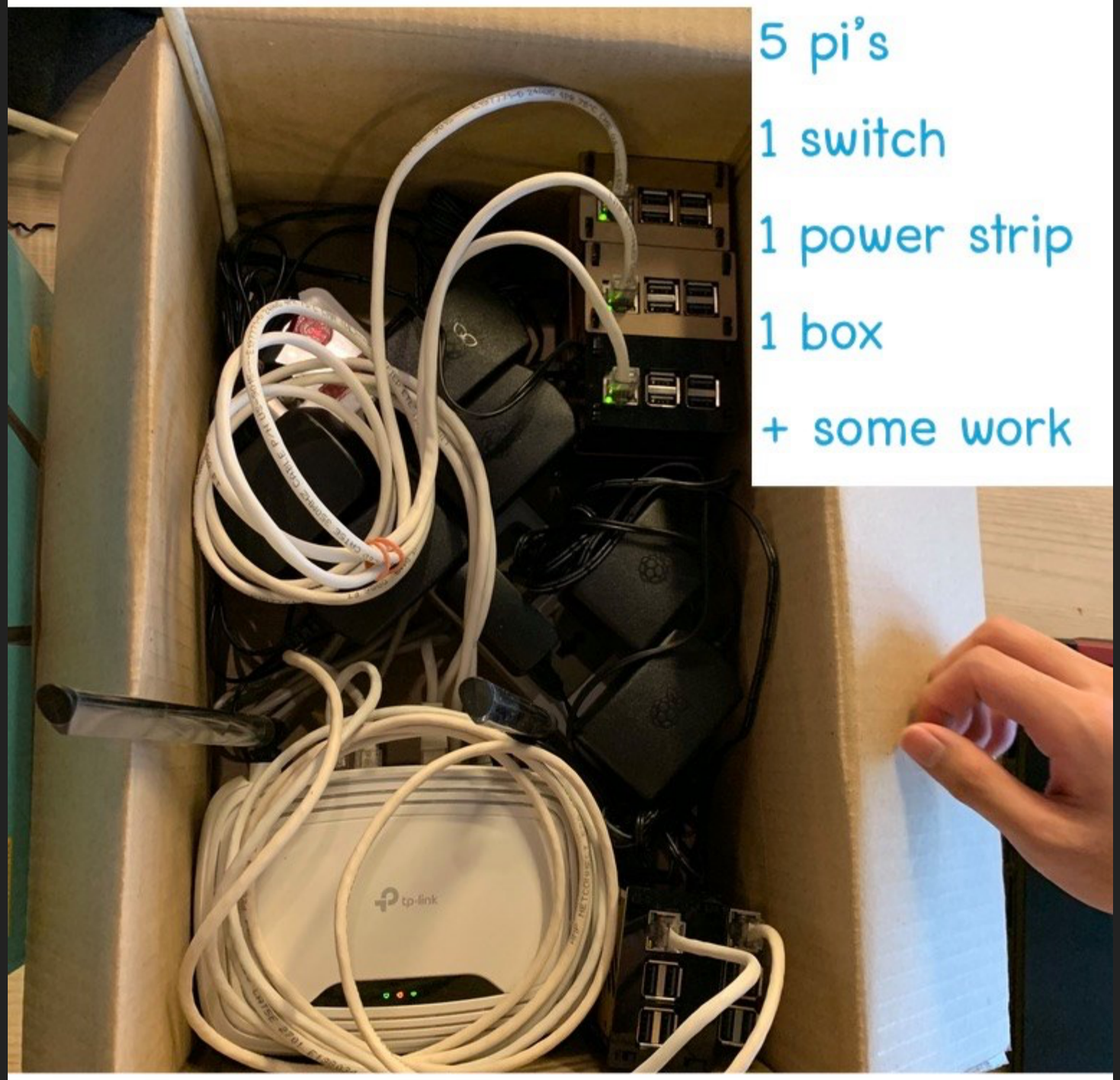
pi kube cluster in a box

CHULA ENGINEERING
Foundation toward Innovation

COMPUTER

Ingredients

- 5 pi's
- 1 switch
- 1 power strip
- 1 box
- + some work



2110415 Software-Defined Systems
offered every first semester

GROUP SIZE

- Form groups of 5 students.

DEVICE CHECKLIST (PER GROUP)

- 1 x Ethernet/WiFi Router TP- Link TL WR841N
- 5x Raspberry Pi Kit (you already have this from HW2)
  - Raspberry Pi 3 B+
  - Power Adapter
  - SD Card
  - Ethernet Cable
  - Ethernet Adapter and USB-C/USB-A Adapter as needed



TEST YOUR PI

1. Turn on your router
2. Connect Pi's to LAN
3. Confirm that your Pi's can use the network (ping, ssh)

OBJECTIVES

- Build a Kuberbetes cluster using all your Raspberry Pi's as nodes (the minimum number of Pi's is equal to the number of members in your group) and your own notebooks as Masters (controllers). The number of controllers is up to you. You may choose to use wired or wireless connectivity, but wired is preferred especially during demo.
- Find/build an application to deploy on your cluster. Your application must use at least ***n* different** container images providing **distinct services** (the minimum number of container images is equal to the number of members in your group). You may choose any application architecture: monolithic or microservice.
- The containers must be dependent on each other, meaning that a client request must be serviced by **at least 2 different containers** before a response is sent back to the client. This requires networking containers on different nodes (yet another application of SDN).
- **Your application must be fault-tolerant** in that if any single node (Raspberry Pi) and its containers go offline, your system will generate replacements and your application will automatically continue to work after the replacements are up and running. (The app can fail until the point where the replacements are running).

APPLICATION CONSIDERATIONS

- If you chose a microservices architecture for your application, the way you break up your services needs to "make sense", i.e., be logical and practical. Do not break things up just for the sake of this project.
- A stateless application is the easiest choice for this project. A stateful application will require more effort. Think about why and design your application wisely.
- You may chose to implement a simple stateless application, as long as the application logic and services make sense.

WHAT TO SUBMIT IN MCV BY NOVEMBER 16, 2026 23:59
LINK TO GITHUB REPO

- Private github repo with README for
  - how to set up your Kubernetes cluster
  - what does your application do
  - how to deploy your application
  - give me (kunwadee) and the TA (theminer3746) access to your repo.
  - Following your instructions, I should be able to create my own cluster and deploy your chosen application on it.

WHAT TO DEMO ON-SITE 13:00-16:00 NOVEMBER 17, 2026

- Demo automatic deployment of your application with all components available in your Github repo
- Demo your application working with successful client requests, and demo that each client request is serviced by 2 or more containers
- Demo fault tolerance in your application by pulling each Raspberry Pi offline, one at a time. Client requests should be successful after containers are online.

STEPS/RUBRIC FOR DEMO ON-SITE 13:00–16:00 NOV 17, 2026

- Have your demo set up and ready before your the start of your demo time slot
- Demo automatic deployment of your application with all components available in your Github repo
- Demo your application working with successful client requests, and demo that each client request is serviced by 2 or more containers
- Demo fault tolerance in your application by pulling a Raspberry Pi offline (instructor will pick a random one). Client requests should be successful after containers are online.
- After demo, sign in and return all equipment
  - Flash all Raspberry Pi's with default Raspian OS
  - Factory reset wireless router

Group members all present	
Confirm master nodes on 2 notebooks, kube cluster is up and running	
Demo automated deployment of application	
Introduce application (understandable presentation)	
Application consists of 4-5 different containers	
Logical design of services/containers	
Each client request touches at least 2 containers	
All 4-5 containers are used across all services	
Show current deployment of containers on nodes	
Demo all client requests successfully	
Randomly pick a pi and note current services running on that pi	
Take pi offline	
Check that the 'replacement' services have been created	
Demo all client requests successfully again	
Factory reset wifi router	
Flash pi's with Raspian OS	
Returned wifi router	
Returned pi's	
Returned power adapters	
Returned ethernet adapters	

THIS COUNTS FOR 100% FOR THE DEMO

HELPFUL HINTS

- Use Ubuntu VM's (at least 2 CPU cores) on your notebooks as Masters.
- Make your very own Kubernetes cluster from Raspberry PI
- https://medium.com/nycdev/k8s-on-pi-9cc14843d43
- https://ubuntu.com/tutorials/how-to-kubernetes-cluster-on-raspberry-pi#1-overview
- This links are a good starting point. Not every thing will work smoothly.